

Plan de Portafolio - Data Engineer con especialización en Telecom

Última actualización: 19 Enero 2025

Objetivo: Transición/Complemento a Data Engineer desde Packet Core Specialist

PERFIL ACTUAL

Habilidades Técnicas

-  Python (OOP, Data Pipelines)
-  SQL & PostgreSQL
-  Apache Airflow (local)
-  Pandas, ETL
-  Power BI (dashboards interactivos)
-  Git (migrar a GitHub)
-  Datacamp: Track Data Engineer completado

Experiencia Relevante

- **Rol actual:** Especialista Packet Core
- **Proyecto destacado:** Data pipeline ETL para logs SGSN-MME
 - Datos: Eventos de señalización (attach, handover, paging)
 - Frecuencia: Lotes cada minuto
 - Destino: PostgreSQL → Power BI dashboard
 - Métricas: Success rates, failure analysis, temporal patterns

Por Aprender

-  Docker & Kubernetes (en proceso)
 -  Cloud (AWS recomendado)
 -  Big Data (Spark, etc.)
 -  GitHub (tengo Git)
-

ESTRATEGIA DE POSICIONAMIENTO

Enfoque recomendado: "Data Engineer con experiencia en Telecomunicaciones"

Ventaja Competitiva

- Conocimiento de dominio telecom (escaso en mercado DE)
- Experiencia real en producción
- Ideal para roles en: Telecom analytics, Network optimization, IoT platforms, 5G analytics

Mercado Objetivo

- Primer objetivo: Data Engineer (mejor demanda/salario/crecimiento)
 - Respaldo: Packet Core Specialist (experiencia actual)
 - Sweet spot: Roles que combinen ambos
-



ROADMAP DE APRENDIZAJE

Fase 1: Fundación (AHORA - próximas 5-6 semanas)

Semana 1-2: Docker & Kubernetes - Datacamp Track (13 horas)

- └─ Curso 1: Containerization and Virtualization Concepts (2h)
- └─ Curso 2: Introduction to Docker (4h)
- └─ Curso 3: Introduction to Kubernetes (3h)
- └─ Curso 4: Intermediate Docker (4h)
- └─ Tomar notas y hacer todos los ejercicios

Semana 3: Práctica Docker - Aplicación a tu proyecto

- └─ Dockerizar cada componente del proyecto
- └─ Crear docker-compose.yml completo
- └─ Testing local y debugging
- └─ Documentación actualizada

Semana 4: Optimización Docker

- └─ Multi-stage builds
- └─ Seguridad de imágenes
- └─ Networking avanzado
- └─ Mejores prácticas (12 Factor App)

Semana 5-6: Kubernetes local (OPCIONAL)

- └─ Minikube o Kind setup
- └─ Crear manifests básicos
- └─ Deploy local del proyecto
- └─ Solo si tienes tiempo extra

Fase 2: Cloud (después de Docker/K8s)

Mes 2-3: AWS Fundamentals

- └─ Curso: Stephane Maarek - Solutions Architect
- └─ Free Tier: EC2, S3, RDS
- └─ Práctica: Deploy proyecto en AWS

Mes 4: AWS Data Engineering

- └─ Glue, Redshift, Kinesis, Athena
- └─ Certificación (opcional): AWS Data Analytics

Fase 3: Big Data (futuro)

Spark, Kafka, Delta Lake, etc.

ESTRUCTURA DE PORTAFOLIO (4 Sprints)

SPRINT 0: Setup & Fundación (2-3 semanas - FLEXIBLE)

Objetivos

- ☒ Definir estrategia de portafolio
- ☐ Crear presencia en GitHub
- ☐ Preparar proyecto base

Tareas

1. GitHub Setup (1-2 horas)

- ☐ Crear cuenta GitHub profesional
- ☐ Configurar perfil:
 - Foto profesional
 - Bio: "Packet Core Specialist | Data Engineer | Python • SQL • Airflow"
 - Location: Bogotá, Colombia
- ☐ Crear primer repositorio: `telecom-network-analytics`

2. Aprender GitHub Básico (3-4 horas)

- ☐ Tutorial Git → GitHub
- ☐ Comandos: push, pull, commit, branch
- ☐ Crear README.md bien estructurado
- ☐ .gitignore para Python

3. Generador de Datos Sintéticos (4-6 horas)

- ☐ Script Python para simular eventos SGSN-MME
- ☐ Eventos: attach, handover, paging, detach, service_request
- ☐ Parámetros configurables:
 - Volumen de datos
 - Ratio éxito/fallo
 - Patrones temporales
 - Anomalías

4. Documentación Inicial (2 horas)

- ☐ README con arquitectura del proyecto
- ☐ Descripción de métricas analizadas
- ☐ Stack tecnológico

Entregables Sprint 0

- ☐ Perfil GitHub activo y profesional
 - ☐ Repositorio `telecom-network-analytics` con README
 - ☐ Script generador de datos funcionando
 - ☐ Primer commit en GitHub
-

SPRINT 1: Proyecto End-to-End Completo (4-5 semanas - FLEXIBLE)

Objetivos

- Refactorizar proyecto actual como showcase profesional
- Integrar Apache Airflow para orquestación profesional
- Código limpio, documentado, reproducible
- Demostrar best practices production-ready

Tareas

1. Refactorización del Pipeline ETL (6-8 horas)

- ☐ Reestructurar código con OOP
- ☐ Separar concerns: extract, transform, load
- ☐ Configuración externa (YAML/JSON)
- ☐ Logging apropiado (Python logging)
- ☐ Manejo de errores robusto

2. Apache Airflow - Aprendizaje (Semana 1-2: ~10-12 horas)

☐ **Datacamp Course:** "Introduction to Airflow in Python" (4h)

- DAGs, Operators, Scheduling
- CLI y Web UI
- Monitoring básico

☐ **Complementos Críticos** (6-8h):

- Docs oficiales: Connections & Hooks (2h)
- Docs oficiales: TaskFlow API (2h)
- Astronomer Best Practices (2h)
- Testing DAGs (2h)

3. Orchestración con Airflow (Semana 3-4: ~12-16 horas)

☐ Setup Airflow local (Docker recomendado)

☐ Configurar PostgreSQL Connection

☐ Implementar DAG completo con:

- Task 1: Generate/Extract events
- Task 2: Data quality validation
- Task 3: Transform & enrich
- Task 4: Load to PostgreSQL (bulk insert)
- Task 5: Calculate KPIs
- Task 6: Anomaly detection

☐ XCom para compartir datos entre tasks

☐ Retry logic con backoff exponencial

☐ SLA monitoring (completar en < 2 min)

☐ Email alerts en failures

☐ Documentación inline del DAG

☐ Screenshots del Airflow UI para portafolio

☐ Testing: Unit tests básicos del DAG

3. Base de Datos PostgreSQL (3-4 horas)

☐ Schema optimizado

☐ Índices apropiados

☐ Scripts DDL documentados

☐ Data quality checks

4. Visualización (4-6 horas)

☐ Dashboard Power BI (exportar PDF/screenshots)

☐ **BONUS:** Crear versión en Streamlit/Plotly Dash

- Más impresionante para portafolio

- Open source
- Deployable en cloud

5. Documentación Técnica Completa (4-6 horas)

- ☐ README detallado:
 - Arquitectura con diagramas
 - Instrucciones de setup paso a paso
 - Descripción de cada componente
 - KPIs y métricas analizadas
- ☐ Código comentado
- ☐ Docstrings en funciones
- ☐ requirements.txt o poetry

6. Testing (opcional pero recomendado)

- ☐ Unit tests básicos
- ☐ Integration tests del pipeline

Entregables Sprint 1

- ☐ Repositorio completo y documentado
 - ☐ Pipeline ejecutable localmente
 - ☐ Dashboard funcionando
 - ☐ Video/GIF demo (opcional pero muy efectivo)
-

SPRINT 2: Cloud-Native con Docker (después de cursos Docker/K8s)

☒ Objetivos

- Modernizar proyecto con containers
- Deploy en AWS
- CI/CD básico
- Demostrar DevOps skills

Tareas

1. Containerización (6-8 horas)

- ☐ Dockerfile optimizado para cada componente:
 - Data generator (multi-stage build)
 - ETL scripts (Python + dependencies)

- Airflow (imagen custom)
 - PostgreSQL (con init scripts)
 - Dashboard Streamlit
- ☐ docker-compose.yml para orquestación local
 - ☐ Optimizar imágenes (multi-stage builds, .dockerignore)
 - ☐ Variables de entorno para configuración
 - ☐ Health checks para cada servicio
 - ☐ Volume mounts para persistencia de datos

Ejemplo docker-compose.yml:

yaml

version: '3.8'

services:

postgres:

image: postgres:14

environment:

POSTGRES_USER: \${DB_USER}

POSTGRES_PASSWORD: \${DB_PASSWORD}

volumes:

- postgres-data:/var/lib/postgresql/data
- ./sql/init.sql:/docker-entrypoint-initdb.d/init.sql

healthcheck:

test: ["CMD-SHELL", "pg_isready -U \${DB_USER}"]

interval: 10s

timeout: 5s

retries: 5

airflow-webserver:

build:

context: ./airflow

dockerfile: Dockerfile

depends_on:

postgres:

condition: service_healthy

ports:

- "8080:8080"

environment:

- AIRFLOW__CORE__SQL_ALCHEMY_CONN=postgres://\${DB_USER}:\${DB_PASSWORD}@postgres/airflow

volumes:

- ./dags:/opt/airflow/dags
- ./logs:/opt/airflow/logs

data-generator:

build: ./data_generator

depends_on:

postgres:

condition: service_healthy

environment:

- DB_HOST=postgres
- DB_PORT=5432

volumes:

postgres-data:

2. AWS Deployment (8-12 horas)

- ☐ Setup AWS Free Tier
- ☐ Arquitectura cloud:
 - RDS PostgreSQL
 - ECS/Fargate para Airflow
 - S3 para logs/backups
 - CloudWatch para monitoring

- ☐ Alternativa serverless:
 - Lambda functions
 - EventBridge/CloudWatch Events
 - Athena para queries

3. Dashboard Cloud (4-6 horas)

- ☐ Streamlit en Cloud Run/App Runner
- ☐ O Power BI Service (si tienes licencia)
- ☐ URL pública accesible

4. CI/CD Básico (4-6 horas)

- ☐ GitHub Actions:
 - Linting (flake8, black)
 - Testing automático
 - Build Docker images
 - Deploy a AWS

5. Documentación Cloud (2-3 horas)

- ☐ Actualizar README con:
 - Arquitectura cloud
 - Costos estimados
 - Instrucciones de deploy
- ☐ Diagrams as code (opcional)

Entregables Sprint 2

- ☐ Proyecto corriendo en AWS
- ☐ URL pública del dashboard
- ☐ Dockerfile y docker-compose.yml
- ☐ CI/CD pipeline funcionando

SPRINT 3: Proyecto Avanzado - Streaming/Real-time (futuro)

✓ Objetivos

- Demostrar versatilidad
- Añadir tecnología trending
- Caso de uso diferente

💡 Opciones de Proyecto

Opción A: Real-time Analytics

- Kafka/Kinesis para streaming
- Spark Streaming para procesamiento
- Dashboard en tiempo real
- Casos: Detección de anomalías en red, alertas automáticas

Opción B: Data Lakehouse

- S3 + Delta Lake/Iceberg
- Medallion architecture (bronze/silver/gold)
- dbt para transformaciones
- Athena/Presto para queries

Opción C: ML Pipeline

- Predicción de fallas de handover
- Feature engineering con Airflow
- MLflow para tracking
- Deployment con SageMaker/simple API

🎯 Entregables Sprint 3

- ☐ Segundo repositorio completo
- ☐ Case study detallado
- ☐ Diferentes tecnologías que Sprint 1-2

📁 ESTRUCTURA GITHUB RECOMENDADA

tu-usuario-github/



telecom-network-analytics/

🌟 PROYECTO ESTRELLA (Sprint 1-2)

```
| | | data_generator/
| | |   generate_telecom_events.py
| | | etl/
| | |   extract.py
| | |   transform.py
| | |   load.py
| | | airflow/
| | |   dags/
| | | sql/
| | |   schema.sql
| | | dashboards/
| | |   powerbi/
| | |   streamlit_app.py
| | | docker/
| | |   Dockerfile
| | |   docker-compose.yml
| | | tests/
| | | docs/
| | |   architecture.md
| | | .github/
| | |   workflows/
| | | requirements.txt
| | | README.md
|
| | streaming-analytics-pipeline/    # Proyecto Sprint 3
| |
| | data-engineering-exercises/      # Ejercicios Datacamp
| |
| | portfolio-website/              # GitHub Pages (opcional)
| |   index.html
```



PLANTILLA README INICIAL

markdown

📶 Telecom Network Analytics Pipeline

🎯 Descripción

Pipeline de análisis end-to-end para eventos de señalización en redes móviles (SGSN-MME).
Procesa logs de attach, handover, paging y otros procedimientos para análisis de KPIs de red.

****Caso de uso:**** Monitoreo y análisis de rendimiento de red móvil en tiempo casi-real.

🏗️ Arquitectura

[Diagrama pendiente]

Componentes:

- ****Data Generator:**** Simulador de eventos de señalización
- ****ETL Pipeline:**** Extracción, transformación y carga con Python
- ****Orchestration:**** Apache Airflow para scheduling
- ****Database:**** PostgreSQL para almacenamiento
- ****Visualization:**** Power BI / Streamlit para dashboards interactivos

🧰 Stack Tecnológico

- Python 3.10+
- Apache Airflow 2.x
- PostgreSQL 14+
- Pandas, SQLAlchemy
- Power BI / Streamlit
- Docker & Docker Compose
- AWS (RDS, ECS, S3)

📊 Métricas Analizadas

KPIs Principales:

- ****Attach Success Rate (ASR):**** % de attach exitosos
- ****Handover Success Rate (HOSR):**** % de handovers sin fallo
- ****Paging Success Rate:**** Tiempo de respuesta promedio
- ****Service Request Success Rate****
- ****Failure Analysis:**** Top causas de fallo por tipo de evento

Análisis Temporal:

- Distribución horaria de eventos
- Tendencias diarias/semanales
- Detección de anomalías

Análisis de Red:

- Performance por celda
- Performance por eNodeB
- Análisis geográfico (opcional)

🚀 Quick Start

Prerequisites

```
```bash
```

Python 3.10+

PostgreSQL 14+

Docker (opcional)

```
```
```

Instalación

```
```bash
```

```
git clone https://github.com/tu-usuario/telecom-network-analytics.git
```

```
cd telecom-network-analytics
```

```
pip install -r requirements.txt
```

```
```
```

Configuración

[Pendiente]

Ejecución

[Pendiente]

📁 Estructura del Proyecto

[Pendiente]

🧪 Testing

[Pendiente]

🗺 Roadmap

- [x] Setup inicial
- [] Generador de datos sintéticos
- [] ETL pipeline básico
- [] Airflow orchestration
- [] Dashboard Power BI
- [] Containerización
- [] AWS deployment
- [] Dashboard Streamlit
- [] CI/CD

📄 Licencia

MIT

👤 Autor

[Tu Nombre]

Packet Core Specialist | Aspiring Data Engineer

✉ email@ejemplo.com | 💼 [LinkedIn](tu-linkedin) | 🐙 [GitHub](tu-github)

⚠️ ****Nota:**** Este proyecto utiliza datos sintéticos simulados. No contiene información real o confidencial.

📊 GENERADOR DE DATOS SINTÉTICOS - Especificaciones

Estructura de Evento

```
python
{
  "timestamp": "2025-01-19T14:23:45.123Z",
  "event_id": "uuid",
  "event_type": "attach | handover | paging | detach | service_request",
  "result": "success | failure",
  "failure_cause": null | "auth_failed | timeout | resource_unavailable | ...",

  # Identifiers (anonimizados)
  "imsi_hash": "hash_value",
  "imei_hash": "hash_value",

  # Network elements
  "cell_id": "CELL_001",
  "enodeb_id": "ENB_042",
  "mme_id": "MME_01",
  "tracking_area": "TA_1",

  # Technical details
  "duration_ms": 245,
  "rat_type": "LTE | 5G | 3G",
  "frequency_band": "B3 | B7 | ...",

  # Event-specific details
  "procedure_details": {
    # Varía según event_type
  }
}
```

Patrones Realistas a Simular

1. Distribución Temporal

- Más eventos en horas pico (8-10am, 12-2pm, 6-9pm)
- Menos eventos en madrugada (2-6am)
- Variación día laboral vs fin de semana

2. Success/Failure Ratios (realistas)

- Attach: 95-98% success
- Handover: 97-99% success
- Paging: 92-95% success
- Service Request: 96-98% success

3. Failure Causes Distribution

Attach failures:

- auth_failed: 40%
- timeout: 30%
- network_congestion: 20%
- other: 10%

Handover failures:

- weak_signal: 50%
- target_cell_congestion: 30%
- timeout: 15%
- other: 5%

4. Anomalías a Simular

- Spike de failures en celda específica (simular outage)
- Degradación gradual de performance
- Eventos de congestión en horas pico

Parámetros Configurables

yaml

```
# config.yaml
data_generation:
  events_per_minute: 1000
  duration_hours: 24

success_rates:
  attach: 0.97
  handover: 0.98
  paging: 0.94

network_elements:
  num_cells: 50
  num_enodebs: 10
  num_mmes: 2

anomalies:
  enabled: true
  outage_probability: 0.001 # 0.1% chance per minute
  congestion_hours: [8, 9, 17, 18, 19]
```

✅ CHECKLIST DE PROGRESO

Sprint 0

- ☐ Cuenta GitHub creada
- ☐ Perfil configurado
- ☐ Repositorio `telecom-network-analytics` creado
- ☐ README inicial subido
- ☐ Generador de datos funcionando
- ☐ Primer push exitoso

Sprint 1

- ☐ ETL refactorizado
- ☐ Airflow DAG documentado
- ☐ PostgreSQL optimizado
- ☐ Dashboard funcionando
- ☐ Documentación completa
- ☐ Video demo (opcional)

Sprint 2

- ☐ Docker funcionando local

- ☐ Deploy en AWS exitoso
- ☐ Dashboard público accesible
- ☐ CI/CD configurado

Sprint 3

- ☐ Segundo proyecto iniciado
 - ☐ Nueva tecnología implementada
 - ☐ Case study completo
-

RECURSOS DE APRENDIZAJE

Docker & Kubernetes

Track Datacamp (PRINCIPAL - 13 horas):

- ☐ Containerization and Virtualization Concepts (2h)
- ☐ Introduction to Docker (4h)
- ☐ Introduction to Kubernetes (3h)
- ☐ Intermediate Docker (4h)

Recursos Complementarios (GRATIS):

- ☐ Docker Docs Oficial: <https://docs.docker.com/get-started/>
- ☐ Play with Docker: <https://labs.play-with-docker.com/> (práctica sin instalar)
- ☐ FreeCodeCamp YouTube: "Docker Tutorial for Beginners" (3h)
- ☐ Kubernetes.io: Tutorial interactivo oficial
- ☐ Airflow in Docker: <https://airflow.apache.org/docs/apache-airflow/stable/howto/docker-compose/>

Herramientas útiles:

- ☐ Hadolint: Linter para Dockerfiles
- ☐ Docker Best Practices: Blog oficial Docker
- ☐ 12 Factor App: Metodología para apps cloud-native

AWS

Cursos recomendados:

1. **Stephane Maarek - Udemy** ★ TOP PICK
 - "Ultimate AWS Certified Solutions Architect Associate"
 - Precio: ~\$15 USD en oferta
 - Duración: ~27 horas
2. **AWS Skill Builder** (Oficial - GRATIS)

- Ruta: "Data Analytics on AWS"
- <https://skillbuilder.aws>

3. A Cloud Guru / Pluralsight

- "AWS Certified Data Analytics Specialty"

Servicios AWS clave para Data Engineering:

- **S3:** Object storage (data lake)
- **RDS/Aurora:** Managed PostgreSQL
- **Redshift:** Data warehouse
- **Glue:** ETL managed service
- **Athena:** SQL queries sobre S3
- **Kinesis:** Real-time streaming
- **EMR:** Managed Spark/Hadoop
- **MWAA:** Managed Workflows for Apache Airflow ★
- **Lambda:** Serverless compute
- **Step Functions:** Workflow orchestration
- **EventBridge:** Event-driven scheduling
- **CloudWatch:** Logs y monitoring

Apache Airflow - Recursos Avanzados:

- ☐ Astronomer Learn: <https://www.astronomer.io/docs/learn/>
- ☐ Astronomer Registry: <https://registry.astronomer.io> (DAG examples)
- ☐ Airflow Best Practices: <https://airflow.apache.org/docs/apache-airflow/stable/best-practices.html>
- ☐ Testing Airflow DAGs: pytest + DAGBag
- ☐ Blog: "Common Airflow Mistakes": Marc Lamberti

GitHub

- ☐ GitHub Skills: <https://skills.github.com>
- ☐ Datacamp: "Introduction to Git"

OPTIMIZACIÓN DE LINKEDIN

Headline sugerido:

Packet Core Specialist | Aspiring Data Engineer | Python • SQL • Airflow • AWS

Sección "About" (ejemplo):

Especialista en Packet Core con 3+ años de experiencia, actualmente expandiendo hacia Data Engineering.

Combino conocimiento profundo en redes de telecomunicaciones (SGSN-MME, señalización móvil) con habilidades técnicas en Python, SQL, y pipelines de datos.

 Proyecto destacado: Pipeline ETL end-to-end para análisis de eventos de señalización móvil (1M+ eventos/día), con orchestration en Airflow y visualización en Power BI.

 Stack: Python | SQL | PostgreSQL | Apache Airflow | Power BI | Git | Docker | AWS

 Buscando oportunidades como Data Engineer, especialmente en telecom analytics o proyectos que combinen mi experiencia en redes con ingeniería de datos.

 Portfolio: [github.com/tu-usuario](#)

Skills a destacar (ordenados):

1. Python
2. SQL
3. Apache Airflow
4. Data Pipelines
5. ETL
6. PostgreSQL
7. Power BI
8. Data Analysis
9. Git
10. Docker

Featured (cuando tengas):

- ☐ Link a repositorio GitHub principal
 - ☐ Artículo sobre tu proyecto en LinkedIn
 - ☐ Certificaciones (cuando las obtengas)
-

CÓMO RETOMAR ESTA CONVERSACIÓN

Cuando vuelvas, comparte:

1. Progreso actual:

- Qué completaste del checklist
- Problemas encontrados
- Preguntas específicas

2. Contexto rápido:

"Hola, estoy siguiendo el plan de portafolio Data Engineer que creamos.
Actualmente estoy en [Sprint X] y he completado [tareas].
Necesito ayuda con [tema específico]."

3. Este documento:

- Menciona que tienes el "Plan de Portafolio - Data Engineer (Telecom)"
- Referencia la sección específica si es necesario

Información de contexto clave para recordar:

- **Background:** Packet Core Specialist → Data Engineer
- **Proyecto principal:** Pipeline ETL para logs SGSN-MME
- **Stack actual:** Python, SQL, Airflow (local), PostgreSQL, Power BI
- **Por aprender:** Docker/K8s (primero), luego AWS
- **Frecuencia datos:** Lotes cada minuto
- **Eventos:** attach, handover, paging, service_request, detach

PRÓXIMOS PASOS INMEDIATOS

Esta semana:

1. **Crear cuenta GitHub** (30 min)
2. **Tutorial Git → GitHub** (1-2 horas)
3. **Crear repositorio** telecom-network-analytics (15 min)
4. **README inicial** (30 min)

Próxima sesión juntos:

- Crear generador de datos sintéticos
 - Estructura del proyecto
 - Configuración inicial
-

MÉTRICAS DE ÉXITO

Para conseguir entrevistas:

- ☐ 2-3 proyectos completos en GitHub
- ☐ README profesionales con diagramas
- ☐ Código limpio y documentado
- ☐ 1+ proyecto con deploy en cloud
- ☐ LinkedIn optimizado con proyectos destacados

Calidad del portafolio:

- ☒ Código funcional y reproducible
 - ☒ Documentación clara
 - ☒ Best practices evidentes
 - ☒ Problema real resuelto
 - ☒ Tecnologías relevantes
-

TIMELINE ESTIMADO

Mínimo viable (3 meses):

- Mes 1: Sprint 0 + Sprint 1
- Mes 2: Docker/K8s + Sprint 2
- Mes 3: AWS + pulir proyectos

Ideal (6 meses):

- Meses 1-2: Sprints 0-1
 - Meses 3-4: Docker/K8s + Sprint 2
 - Meses 5-6: AWS + Sprint 3 + aplicar a empleos
-

Última actualización: 19 Enero 2025

Versión: 1.0

💡 **Recuerda:** La consistencia > intensidad. Mejor 1 hora diaria que 10 horas un solo día.

🎯 **Focus:** Calidad sobre cantidad. Un proyecto excelente vale más que tres mediocres.

🚀 **Momentum:** Empieza pequeño pero empieza YA. El primer commit es el más importante.